11 POLYMORPHISM • THEORY

03 object slicing and name hiding

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

03 — Object Slicing and Function Name Hiding

1. Object Slicing

If you assign or pass a `derived` object **by value** into a `base` variable (`base b = d;`), only the `base` subobject portion of `d` is copied; all `derived` -specific data members and the `derived` vptr are **sliced off**! Runtime polymorphism **never** works when passing objects by value — you must use a pointer (`base*`) or reference (`base&`).

2. Function Name Hiding (`using base::func;`)

If class `super` has `void show(int x)` and class `sub` defines `void show()` (with 0 parameters), `sub::show()` **hides** all overloaded versions of `show` in `super` ! Calling `sub_obj.show(10)` fails to compile unless `sub` includes `using super::show;` .